AI 對話摘要：我如何和 AI 一起做出這個 LINE Bot

> 這份摘要依 GitHub commit 紀錄、程式碼與實際除錯過程整理。交件前請依你的真實情況補修姓名、時間與細節；不要把它當成「AI 全自動完成」的證明，因為真正讓專案跑起來的是每一次手動測試與修正。

---

1. 起點：我想做一個「真的會用」的 LINE Bot

我一開始的題目很發散：想做一個 LINE Bot，但不想只是「回固定文字」或「接一個昂貴的 API」。我希望它至少能做到兩件事：

1. 回答需要查資料的問題，而且答案要附來源。
2. 順手把生活待辦記下來。

我給 AI 的第一版需求大概是：

> 「我想做一個 LINE Bot，使用者在 LINE 裡問問題，Bot 會先用 Ollama 在本機回答；如果問題需要最新資訊，就上網搜尋，回答最後附來源。其他訊息可以存成待辦。請幫我規劃架構，先不要一次寫完。」

AI 先提出了「LINE Webhook → Flask → Ollama」的基本架構，並提醒我 LINE 需要公開的 HTTPS 端點，本機的 `localhost` 不能直接給 LINE 打。這一句話決定了後來的整個設計方向。

---

2. 第一輪：先做出能跑的最小版本

AI 產生了 `line_bot_local.py`：用 Flask 接 Webhook、用 `line-bot-sdk` 驗證簽章、用 DuckDuckGo 搜尋、用 Ollama 回答。這個版本的本機測試可以跑，但很快遇到三個問題：

1. **DuckDuckGo 回傳 0 筆**：AI 原本假設直接抓 HTML 就好，但實際上有反機器人頁面，解析不到結果。
2. **Gemma 2 9B 太慢**：在這台純 CPU 電腦上生成很久，LINE 會逾時。
3. **LINE 收不到回覆**：Webhook 有收到，但同步等模型算完才回，超過 LINE 的等待時間。

這時候我學到：**AI 給的「看起來合理」的程式，不等於在真實環境可跑。** 我開始把錯誤訊息、終端機 log、實際回應貼回對話，而不是只說「壞掉了」。

---

3. 第二輪：嘗試雲端，最後改成混合架構

AI 建議了兩條路：

- **Cloudflare Worker + Gemini API**：不用本機電腦，但要 API key 與額度。
- **Google Apps Script + 本機 Ollama**：雲端只收訊息，推論留在家裡。

我先做了 Gemini 版，程式比較短，但需要申請 key，而且我原本就想試本地模型，所以最後選了混合架構：

```
LINE → Apps Script → ngrok → Flask → Ollama → LINE Reply API
```

AI 在這一步幫我拆出 `Code.gs`（Webhook）與 `relay_server.py`（搜尋 + 模型），也說明了為什麼 Apps Script 不能直接連 `localhost`。但它同時給了一個很容易踩的坑：把 token 直接寫在程式碼最上面。這個做法在 demo 時很方便，卻不應該進 repo。

---

4. 第三輪：真正的除錯馬拉松

4.1 搜尋來源不穩

DuckDuckGo 被擋之後，AI 提出改用 **Google News RSS + 中文 Wikipedia API**，兩個都免金鑰。這個改動讓搜尋成功率穩定很多，但也帶來新問題：

- Google News 的查詢字串如果沒清理乾淨，會回傳 404 或 0 筆。
- 新聞標題後面常帶「 - 媒體名」，畫面很亂。
- Google News 的文章連結又長又會轉址，不適合直接貼給使用者。

最後的解法是：清掉「查 / 請問 / 有多高」等口語詞、用媒體首頁當來源、跳過社群連結、限制前 3～4 筆。

4.2 LINE 逾時

AI 提出「Apps Script 先回 200，Flask 算完再直接呼叫 LINE Reply API」的非同步做法。這是整個專案最關鍵的修正之一。改完之後，使用者不會再看到 Webhook 逾時，但新的問題是：如果 Flask 沒有 token，就沒辦法自己回 LINE。因此 token 改成用環境變數傳給 Flask，並且在啟動時印出「已設定 / 未設定」，方便排查。

4.3 模型太慢

原本用 `gemma2:9b`，生成時間太長。AI 建議改成 `gemma2:2b`，並加上：

- 啟動時先 `warm_up()`，避免第一次提問等太久。
- `num_predict=220`、`num_ctx=2048`，限制生成量。
- 只把前 3 筆搜尋結果放進 prompt。

改完後，暖機查詢大約 5～6 秒；冷啟動約 16 秒。這仍然不是「即時」，但對 LINE 來說可以接受。

4.4 部署網址與權限

Apps Script 的部署有「測試版」與「正式版」兩種網址。我一開始貼錯，LINE Verify 出現 302；後來把「存取權」設成「任何人」才成功。AI 一開始沒有分清楚這兩種網址，只說「貼部署 URL」，這讓我多花了時間。後來我在 README 寫清楚：**要用網頁應用程式的正式部署網址，不是測試用的開發網址。**

4.5 亂碼與 token

Windows 終端機的中文輸出變成亂碼，AI 幫我加上 UTF-8 重新設定。另一個更嚴重的問題是：本機啟動腳本裡有真實 token。AI 幫我把檔案加進 `.gitignore`、把 token 改成環境變數，並在交件前加了一個掃描腳本。**如果 token 曾經出現在任何公開地方，正確做法不是刪掉就好，而是立刻重新發行。**

---

5. 最後一輪：讓它在課堂上穩定跑

距離 demo 越近，我越擔心「當天網路不穩、ngrok 斷線、token 過期」。所以我又請 AI 幫我做一個**不需要 LINE 的展示頁**：

- 同一個 `/ask` 流程，但用瀏覽器顯示。
- 手機版聊天 UI，可以直接點「查 玉山有多高」、「買牛奶」。
- 右側顯示搜尋、生成、回覆的流程與耗時。
- 待辦寫入本機 `todo.txt`，不用 Google Sheet 也能展示。

這個展示頁後來變成整個專案最適合 demo 的部分：它把「AI 查資料」與「待辦捕捉」兩個價值放在同一個畫面，而且不用在課堂上冒著 token 外洩的風險。

---

6. 我用了哪些 prompt？

有效的 prompt

目的	Prompt 寫法
先規劃再寫	「先不要寫程式，先列出架構、每個元件的責任、以及最可能失敗的三個地方。」
縮小問題	「只改 `askRelay`，不要動其他檔案；問題是 Apps Script 回 302。」
要求可驗證	「給我可以在本機跑的測試指令，以及預期輸出。」
安全檢查	「掃描這個 repo，找出 token、Bearer、ngrok authtoken、Apps Script 部署網址；只掃 git 追蹤的檔案。」
文件整理	「依 commit 紀錄與程式碼，整理一份包含動機、架構、測試、限制的報告大綱。」

沒那麼有效的 prompt

問題	為什麼沒效	後來怎麼改
「幫我做一個 LINE Bot」	太發散，AI 會自己補一堆假設	補上「本機 Ollama、要搜尋、要附來源、其他訊息存待辦」
「壞了，幫我修」	沒有 log，AI 只能猜	貼完整錯誤訊息、作業系統、執行指令
「幫我美化」	不知道目標與限制	指定「給 90 秒課堂 demo、要能離線展示、不要 token」
「順便幫我部署」	涉及帳號與權限，AI 不應該代做	自己部署，AI 只提供檢查清單

---

7. AI 做得好與不好的地方

做得好

- 快速產生可運作的第一版，讓我不必從空白檔案開始。
- 把大問題拆成「Webhook / 搜尋 / 模型 / 回覆」幾個區塊。
- 看得懂錯誤訊息，能提出替代方案（例如 DuckDuckGo → RSS + Wikipedia）。
- 幫我補齊 README、測試、展示頁與資安檢查。
- 在我卡住時提供「先確認哪一個環節」的除錯順序。

不夠好

- 一開始把 token 寫死在程式碼裡。
- 假設 DuckDuckGo 一定抓得到，沒有先驗證。
- 沒有主動區分 Apps Script 的測試網址與正式網址。
- 產生的程式有時會用到不存在的函式或錯的 API 參數。
- 很容易「看起來完成了」，但沒有實際跑過。

---

8. 如果重來一次，我會怎麼做？

1. **先寫失敗清單**：網路、token、逾時、搜尋、模型速度，先想好每個環節的 fallback。
2. **每個小功能都先跑一次**：不要等整個系統寫完才測試。
3. **一開始就用環境變數與 `.gitignore`**：不要等到要交件才處理 token。
4. **先做展示頁，再做 LINE**：展示頁讓我能快速驗證核心流程，也不會被 LINE 的憑證卡住。
5. **要求 AI 給測試**：沒有測試的程式碼，等於沒有完成。

---

9. 一句話總結

AI 幫我把「想法」變成「可以跑的原型」，但把原型變成「可以交的專案」的是：**實際執行、看 log、縮小問題、補測試，以及對資安保持偏執。**